



Guide 3: The 4 Best Machine Learning Models Explained

Project: Personal Finance Classification Using Machine Learning

Purpose: Plain-English, comprehensive guide explaining **HOW**, **WHY**, and **WHEN** each of the 4 best classification algorithms works in this project.

The 4 Best Models at a Glance

Rank	Model Name	Model Paradigm	5-Fold CV Accuracy	Test Accuracy	Macro F1-Score	Inference Latency	Primary Strength
1 ★	Decision Tree	Rule-Based Tree	99.99% ± 0.02%	99.98%	99.98%	Instantaneous (<1 ms)	Best Overall: Maximum clarity, zero black-box mystery, instant rules
2	Random Forest	Bagging Ensemble	99.99% ± 0.02%	99.98%	99.98%	Fast (~3 ms)	Ultra-Robust: Averages 100 diverse trees to eliminate variance
3	Gradient Boosting	Boosting Ensemble	99.99% ± 0.02%	99.98%	99.98%	Moderate (~6 ms)	Error-Hunter: Sequentially fixes mistakes of previous trees
4	Logistic Regression	Multinomial Linear	99.62% ± 0.12%	99.62%	99.64%	Ultra-Fast (<0.5 ms)	High-Throughput: Transparent linear equations, calibrated probabilities

1. Decision Tree Classifier (★ Best Performing Model)

Real-World Analogy

Think of the children's game "20 Questions" or a medical diagnosis flow chart:

"Does the patient have a fever? Yes → Is oxygen below 95%? Yes → Admitted to ICU."

A **Decision Tree** asks a sequence of simple true/false questions about your financial numbers. Each question splits the data into smaller, cleaner groups until it reaches a final decision: **Saver**, **Balanced**, or **High-Spender**.

HOW does it work mathematically?

1. **Root Node (First Question):** The algorithm tests every single feature and every possible threshold value to find which question cleanest splits the data.
2. **Gini Impurity Criterion (G):** To measure "cleanliness", the tree computes the Gini Impurity:

$$G = 1 - \sum_{i=1}^C p_i^2$$

Where p_i is the fraction of records belonging to class i . - If a bucket contains 33% Savers, 33% Balanced, and 33% High-Spenders, $G \approx 0.67$ (**Messy / High Impurity**). - If a bucket contains 100% Savers, $G = 1 - (1.0)^2 = 0.0$ (**Pure / Zero Impurity**).

3. **Information Gain:** The tree chooses the question that produces the largest **drop in impurity**:

$$\Delta G = G_{\text{parent}} - \left(\frac{N_{\text{left}}}{N} G_{\text{left}} + \frac{N_{\text{right}}}{N} G_{\text{right}} \right)$$

4. **Our Hyperparameters:** `python DecisionTreeClassifier(max_depth=6, random_state=42)` We capped `max_depth=6`. This means the tree is restricted to at most 6 question levels deep, guaranteeing it captures the true underlying logic without memorizing noisy individual rows.

WHY did it perform so well in this project?

- **Test Accuracy: 99.98%** (CV: **99.99% ± 0.02%**).
- **Why it crushed the data:** Financial archetypes are separated by distinct thresholds. For example:
 - If `savings_rate >= 32%` and `expense_to_income <= 68%` → Almost certainly a **Saver**.
 - If `savings_rate < 15%` and `debt_to_income > 10%` → Almost certainly a **High-Spender**.
- Decision trees are masters of finding axis-aligned, orthogonal threshold splits. In ratio space, it separated the three classes with near-zero error.
- **Why it is our #1 Pick:** It delivers equal accuracy to complex ensembles (100 trees), but uses only a **single tree** with instantaneous execution and complete explainability.

WHEN should you use Decision Trees?

- When you need a **100% interpretable model** that you can draw on a whiteboard and explain to a customer, auditor, or bank regulator.
- When working with tabular business data containing categorical and numerical thresholds.
- When inference speed must be virtually instantaneous.

WHEN should you avoid Decision Trees?

- When your data has diagonal or circular decision boundaries (a single tree has to approximate smooth curves with jagged stair-steps).
- When you have noisy data without setting `max_depth` (an unconstrained tree will overfit to every outlier).

2. Random Forest Classifier

Real-World Analogy

Imagine you want to predict who will win an election. If you ask just **one voter** (a single Decision Tree), they might give a biased or quirky answer. But if you survey **100 diverse people from different walks of life** (a Random Forest) and take a majority vote, individual biases cancel out, and the collective wisdom is remarkably accurate. That is the **Wisdom of the Crowds**.

HOW does it work mathematically?

A Random Forest is a **Bagging (Bootstrap Aggregating)** ensemble of many individual decision trees.

1. **Bootstrapping (Random Data Sampling):** - If our training set has 16,000 rows, the algorithm builds 100 different datasets by randomly picking 16,000 rows **with replacement** (some rows are picked twice, some are left out). - Each of the 100 trees is trained on a slightly different version of the dataset.
2. **Random Feature Subspace Sampling:** - At every single split node in every tree, the algorithm does not consider all 18 features. It only considers a random subset of $\sqrt{p}$ features (e.g., $\sqrt{18} \approx 4$ random features). - This prevents one dominant feature (like `savings_rate`) from dominating every single tree, forcing other trees to discover hidden secondary patterns.
3. **Majority Voting (Aggregation):** - When a new user profile arrives, all 100 trees independently classify it. - If 97 trees vote "Saver", 2 vote "Balanced", and 1 votes "High-Spender", the forest outputs **"Saver" with 97% confidence**.
4. **Our Hyperparameters:** `python RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)`

WHY did we use it?

- **Test Accuracy: 99.98%** (CV: **99.99% $\pm$ 0.02%**).
 - It provides supreme stability. Even if a user enters unusual or slightly noisy combinations (e.g., very high income but strange transit spend), the forest averages out anomalies and makes a robust prediction.
 - It produces reliable **Gini Feature Importance** rankings across all 100 trees, proving that savings rate and expense ratio account for over 70% of classification power.
-

WHEN should you use Random Forests?

- Default choice for tabular datasets in production when you want high accuracy out of the box with minimal hyperparameter tuning.
- When you want to eliminate the high variance and overfitting tendencies of a single decision tree.

WHEN should you avoid Random Forests?

- When memory is severely constrained (saving 100 trees takes ~700 KB on disk, compared to 2 KB for a single tree).
 - When you must produce a single, simple diagram of the decision logic.
-

3. Gradient Boosting Classifier

Real-World Analogy

Imagine a team of 80 golfers trying to hit a ball into a hole: - Golfer 1 takes a swing and lands 30 meters left of the pin. - Golfer 2 does **not** try to hit the whole shot again; Golfer 2 specifically studies Golfer 1's 30-meter mistake and hits a correction shot. - Golfer 3 studies whatever small residual error remains after Golfer 2, and refines it further. - By the 80th golfer, the ball drops directly into the cup.

In **Gradient Boosting**, trees do not work independently in parallel (like Random Forest). They work **sequentially in a relay race**, where each new tree specifically trains on the **errors (residuals)** made by the previous trees.

HOW does it work mathematically?

1. **Initial Prediction:** Starts with a base prediction (the overall log-odds of each class).
2. **Compute Residual Errors:** Calculates the gradient of the loss function (how far the prediction is from the actual label):

$$r_{ik} = y_{ik} - p_k(x_i)$$

Where $y_{ik} \in 0, 1$ is the actual class indicator, and $p_k(x_i)$ is the predicted probability.

3. **Train a Shallow Tree on Residuals:** A small decision tree (weak learner) is fitted specifically to predict these negative gradient residuals r_{ik} .
4. **Learning Rate Shrinkage (η):** Rather than blindly adding the whole new tree's prediction, it scales the contribution by a learning rate ($\eta = 0.1$):

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

This deliberate "shrinkage" prevents the model from jumping aggressively toward noisy outlier points.

5. **Our Hyperparameters:** `python GradientBoostingClassifier(n_estimators=80, max_depth=4, learning_rate=0.1, random_state=42)`

WHY did we use it?

- **Test Accuracy: 99.98%** (CV: **99.99% $\pm$ 0.02%**).
- Gradient Boosting is renowned across Kaggle competitions and FinTech platforms (credit scoring, anti-fraud) as the gold standard for structured tabular data.
- It is extraordinarily effective at resolving "fuzzy boundary" edge cases (people right on the border between Balanced and High-Spender).

WHEN should you use Gradient Boosting?

- When you are competing for the highest possible fractional percentage point of predictive accuracy.
- When predicting financial credit defaults, loan approvals, or insurance claims on tabular datasets.

WHEN should you avoid Gradient Boosting?

- When training time is limited on massive multi-million row datasets (sequential training cannot be easily parallelized like Random Forest).
- When your data has many mislabeled rows or severe outliers, as boosting can over-focus on correcting pure noise.

4. Logistic Regression

Real-World Analogy

Think of a bank's classic credit scorecard. Each factor gets assigned points: - Savings Rate > 30% → **+40 points** - Debt-to-Income > 20% → **-30 points** - Discretionary Spending > 35% → **-20 points**

You add up the weighted points (z). If your score is high, you land in Saver; if in the middle, Balanced; if low, High-Spender. **Logistic Regression** is the mathematical formalization of this intuitive scoring system.

HOW does it work mathematically?

Despite the word "Regression" in its name, Logistic Regression is a **classification algorithm**.

1. **Linear Combination:** For each of the 3 classes ($k \in 0, 1, 2$), it calculates a linear score:
$$\mathbf{z}_k = \mathbf{w}_k^T \mathbf{x} + b_k = w_{k,1} x_1 + w_{k,2} x_2 + \dots + w_{k,p}$$
2. **Softmax Function (Multi-Class Probability):** Converts raw scores z_0, z_1, z_2 into smooth probabilities that sum to 100%:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^3 e^{z_j}}$$

3. **Loss Function (Categorical Cross-Entropy):** Penalizes confident incorrect predictions:

$$L(\mathbf{w}) = - \sum_{i=1}^N \sum_{k=1}^3 y_{ik} \log P(y_i = k \mid \mathbf{x}_i) + \frac{1}{2C} \|\mathbf{w}\|^2$$

4. **L2 Regularization ($C = 1.0$):** Adds a penalty on large weights to prevent any single variable from blowing up.
 5. **Our Hyperparameters:** `python LogisticRegression(max_iter=1000, C=1.0, random_state=42)`
-

WHY did we use it?

- **Test Accuracy: 99.62%** (CV: **99.62% ± 0.12%**).
 - **The Linear Miracle:** Achieving 99.62% accuracy with a purely linear model is a massive triumph. It proves that our **domain feature engineering** was so effective that the three archetypes are virtually linearly separable in ratio space!
 - **Inference Speed:** Runs in under **0.5 milliseconds** ($O(1)$ constant time).
 - **Direct Interpretability:** The model coefficients directly show the exact positive or negative impact of each dollar and percentage point.
-

WHEN should you use Logistic Regression?

- High-frequency production environments (e.g., real-time payment gateways processing 10,000 transactions/second).
- Heavily regulated banking sectors (e.g., US Fair Lending Act, European GDPR) where decisions must be mathematically explainable without black-box approximations.

WHEN should you avoid Logistic Regression?

- When relationships between features are heavily non-linear and you haven't engineered ratios to linearize them.

Comprehensive Head-to-Head Comparison

Comparison Metric	Decision Tree	Random Forest	Gradient Boosting	Logistic Regression
Test Accuracy	99.98%	99.98%	99.98%	99.62%
5-Fold CV Stability	99.99% ± 0.02%	99.99% ± 0.02%	99.99% ± 0.02%	99.62% ± 0.12%
Inference Latency	< 1 ms	~3 ms	~6 ms	< 0.5 ms
Model File Size	2.3 KB	681 KB	167 KB	1.3 KB
Requires Feature Scaling?	No	No	No	Yes (StandardScaler)
Interpretability	Highest (Visual Tree)	Moderate (Importance)	Moderate (Importance)	High (Linear Weights)
Overfitting Risk	Medium (if unpruned)	Very Low	Low (with shrinkage)	Very Low
Training Speed	Fastest (~0.1s)	Moderate (~2.5s)	Slower (~12s)	Fast (~0.4s)
Best Production Use	Live Interactive UI	Enterprise Analytics	High-Stakes Risk Engine	High-Frequency API

💡 Why were KNN and SVM Dropped?

In earlier iterations of this project, **K-Nearest Neighbors (KNN)** and **Support Vector Machines (SVM)** were benchmarked alongside these 4 models. Here is why they were eliminated:

1. **KNN (Dropped at 93.08% Accuracy):** - KNN is an *instance-based learner* that doesn't actually learn a model; it memorizes the entire dataset and computes Euclidean distances against all 16,000 training points on every single prediction. - It had the lowest accuracy (93.08%) and high memory consumption ($O(N)$ inference latency).
2. **SVM (Dropped at 98.72% Accuracy):** - While SVM performed reasonably well (98.72%), computing probabilities required expensive Platt scaling, training was computationally heavy ($O(N^2)$ to $O(N^3)$), and its RBF kernel mapping is an uninterpretable mathematical black box.

By keeping **Decision Tree, Random Forest, Gradient Boosting, and Logistic Regression**, the project retains the absolute highest accuracy (**99.62% to 99.98%**), the fastest inference times, and complete technical interpretability.

Summary: You now possess a comprehensive, viva-ready understanding of the mathematical foundations, performance trade-offs, and practical deployment reasons for all 4 models.